FAULT DIAGNOSIS IN INDUSTRIAL SYSTEM

K. Pragnya - 2520030430

G. Sai Lasya – 2520030538

V. Keathana - 2520090236

PYTHON CODE

```
"""

from collections import deque
from dataclasses import dataclass
from typing import Dict, List
import math

# ======================================================
# CO1: Agent Model (PEAS), State Representation
# ======================================================

@dataclass
class MachineState:
temperature: float
vibration: float
pressure: float
current: float

# Knowledge representation using RULES (CO1)

fault_rules = {
"Bearing Fault":
lambda s: s.vibration > 7,

"Overheating":
lambda s: s.temperature > 85,

"Leakage":
lambda s: s.pressure < 20,

"Electrical Fault":
lambda s: s.current > 50
}
```

```
# ========================================================
# CO2: BFS Search for Fault Diagnosis
# ========================================================

def bfs_diagnosis(state):

queue = deque()
queue.append(state)

visited = set()

while queue:

current = queue.popleft()

key = (
current.temperature,
current.vibration,
current.pressure,
current.current
)

if key in visited:
continue

visited.add(key)

faults = []

for fault, rule in fault_rules.items():
if rule(current):
faults.append(fault)

return faults

return ["Normal"]
```

```
# ========================================================
# CO3: Constraint Satisfaction Problem (CSP)
# ========================================================

def check_constraints(state):

violations = []

# Constraint propagation intuition
if state.temperature > 90 and state.current > 55:
violations.append(
"Temperature-Current constraint violated"
)

if state.vibration > 8 and state.pressure < 25:
violations.append(
"Vibration-Pressure constraint violated"
)

return violations
# ========================================================
# CO5: Bayesian Inference
# ========================================================

def bayes_fault_probability(temp):

# Prior probability
prior_fault = 0.1

# Likelihood P(Evidence | Fault)
if temp > 85:
likelihood = 0.8
else:
likelihood = 0.2

evidence = 0.3
```

```
posterior = (
likelihood * prior_fault
) / evidence

return round(posterior, 3)
# ======================================================
# CO4: Utility Function
# ======================================================

def utility(state):

score = 100

score -= abs(state.temperature - 70)
score -= state.vibration * 2
score -= abs(state.pressure - 30)

return score

# ======================================================
# CO6: Hybrid AI System
# Search + CSP + Probability + Decision Logic
# ======================================================

def diagnose(state):

print("\n----- INDUSTRIAL DIAGNOSIS -----")

# CO2 Search
faults = bfs_diagnosis(state)

# CO3 CSP
constraints = check_constraints(state)

# CO5 Probability
probability = bayes_fault_probability(
state.temperature
```

```
)

# CO4 Decision Utility
utility_score = utility(state)

print("Detected Faults:", faults)

print("\nConstraint Violations:")
if constraints:
for c in constraints:
print("-", c)
else:
print("None")

print("\nFault Probability:", probability)

print("Utility Score:", utility_score)

# Decision Logic (CO6)
if probability > 0.5:
print("\nDecision: Maintenance Required")
else:
print("\nDecision: Continue Operation")
# ========================================================
# Main Program
# ========================================================

machine = MachineState(
temperature=95,
vibration=8.5,
pressure=18,
current=60
)

diagnose(machine)
```

Sample Output

```
----- INDUSTRIAL DIAGNOSIS -----

Detected Faults:
['Bearing Fault', 'Overheating', 'Leakage', 'Electrical Fault']

Constraint Violations:
- Temperature-Current constraint violated
- Vibration-Pressure constraint violated

Fault Probability: 0.267
Utility Score: 16.0
Decision: Continue Operation
```